

Web Development

Labo 12

Jari Rhellam

Date Objecten

Huidige datum en tijd ophalen

1. `let today = new Date();`

tijd objecten aanmaken

`new Date(value);`

`// op basis van timestamp`

`new Date(dateString);`

`// op basis van een string (ISO-8601 aanbevolen)`

`new Date(year, month[, day, hour, minute, second, millisecond]);`

`// afzonderlijke onderdelen`

voorbeeld:

```
let verjaardag1 = new Date('December 17, 2025 03:24:00');
let verjaardag2 = new Date('2025-12-17T03:24:00'); // ISO-formaat
let verjaardag3 = new Date(2025, 11, 17); // maand 0-indexed!
let verjaardag4 = new Date(2025, 11, 17, 3, 24, 0);
```

Datum uitlezen:(getters)

```
event.getDate(); // dag van de maand (1-31)
event.getMonth(); // maand (0-11)
event.getFullYear(); // jaar
event.getHours(); // uren
event.getMinutes(); // minuten
event.getSeconds(); // seconden
```

Datums instellen(setters)

```
event.setDate(25);
event.setMonth(0); // januari
event.setFullYear(2025);
```

Datum als text weergeven

```
event.toString(); // "Fri Apr 10 2020 23:15:30 GMT+0200"
event.toISOString(); // "2020-04-10T12:48:00.000Z" (UTC)
event.toDateString(); // "Thu Apr 10 2025"
event.toTimeString(); // "12:10:40 GMT+0200"
```

Zelf datum opbouwen voor UI

```
let event = new Date('2025-04-10T12:10:30');
console.log(event.getDate() + "-" + (event.getMonth()+1) + "-" + event.getFullYear() +
  " " + event.getHours() + ":" + event.getMinutes() + ":" + event.getSeconds());
// Output: "10-4-2025 12:10:30"
```

Opdracht dagen op de wereld

```
// 1. Maak een Date object voor je geboortedag
// let geboortedag = new Date(jaar, maand, dag);
// Let op: maanden zijn 0-indexed, dus 0 = januari, 11 = december
let geboortedag = new Date(2005, 11, 17);
// voorbeeld: 17 december 2005

// 2. Huidige datum ophalen
let vandaag = new Date();
// maakt een Date object voor het huidige moment

// 3. Bereken het verschil in milliseconden
// Je kunt twee Date objecten van elkaar aftrekken; resultaat is in milliseconden
let verschilMs = vandaag - geboortedag;

// 4. Zet milliseconden om naar dagen
// 1 dag = 1000 ms * 60 s * 60 min * 24 uur
let verschilDagen = Math.floor(verschilMs / (1000 * 60 * 60 * 24));

// 5. Resultaat weergeven
console.log("Aantal dagen sinds geboortedag:", verschilDagen);
```

Extra tip voor je labo:

- Math.floor() zorgt dat je een heel getal krijgt (geen decimalen).
- Als je het resultaat ook in de pagina wilt tonen i.p.v. de console, kan je document.write() of innerText gebruiken.

Eigen object maken in javascript

1. Object aanmaken

```
// Een leeg object maken  
let student1 = {};
```

2. Properties toevoegen

```
// Je kan gewoon eigenschappen toevoegen  
student1.voornaam = "Jan";  
student1.geboorteDatum = new Date("1989-04-21");  
  
// Object binnen een object (nested object)  
student1.adres = {};  
student1.adres.gemeente = "8500";  
  
console.log(student1.voornaam + " woont in " + student1.adres.gemeente);
```

In JavaScript kan je dynamisch properties toevoegen (zonder class).

3. Properties verwijderen

```
delete student1.voornaam;
```

4. Niet bestaande properties

```
console.log(student1.postcode); // undefined
```

5. Twee manier om properties te gebruiken

```
// Dot notation (meest gebruikt)  
student1.voornaam;  
  
// Bracket notation  
student1["voornaam"];
```

Beide zijn hetzelfde, maar [] gebruik je in speciale gevallen:

6. Wanneer gebruik je [] ?

a) *Speciale namen*

```
student1["thuis adres"] = "Kortrijk"; // spatie → moet met []
```

b) *Property uit variabele*

```
let propertyNaam = "voornaam";  
console.log(student1[propertyNaam]); // gebruikt variabele
```

Object literals

Een **object literal** is een **snelle manier om een object te maken met properties**.

Zonder object literal (lang):

```
let student = {};  
  
student.voornaam = "Jan";  
  
student.familienaam = "Janssens";  
  
student.geboorteDatum = new Date("1993-12-31");
```

- *Met object literal (kort en beter):*

```
let student = {  
  voornaam: "Jan",  
  familienaam: "Janssens",  
  geboorteDatum: new Date("1993-12-31")  
};
```

Belangrijk:

- Gebruik : tussen naam en waarde
- Gebruik , tussen properties
- Laatste property → **geen komma nodig**

Values mogen ook variabelen zijn

```
let student = {  
  voornaam: txtVoornaam.value,  
  familienaam: txtFamilienaam.value,  
  geboorteDatum: new Date(txtGeboorteDatum.value)  
};
```

Complex object (met objecten + arrays)

```
let student = {  
  voornaam: "Jan",  
  familienaam: "Janssens",  
  geboorteDatum: new Date("1993-12-31"),  
  
  adres: { // genest object  
    straat: "Kerkstraat 13",  
    postcode: "8500",  
    gemeente: "Kortrijk"  
  },  
  
  isIngeschreven: true, // boolean  
  
  namenVanExen: ["Sofie", "Berta", "Philip"], // array  
  
  aantalAutos: 2  
};
```

(object literals)

- Snelle manier om objecten te maken
- Bestaat uit **key: value** paren
- Kan **objecten + arrays bevatten**
- Veel gebruikt in praktijk

JSON

JSON = JavaScript Object Notation

- Lijkt op een object literal
- Wordt gebruikt om **data op te slaan of te versturen** (bv. API's)

Voorbeeld;

```
{  
  "voornaam": "Jan",  
  "familienaam": "Janssens",  
  "leeftijd": 25  
}
```

Verschil met JavaScript object:

JavaScript object	JSON
keys zonder quotes mogelijk	keys moeten in quotes
mag functies bevatten	geen functies
flexibel	puur data

- JSON gebruiken in JavaScript

```
// Object → JSON  
let jsonString = JSON.stringify(student);  
  
// JSON → Object  
let obj = JSON.parse(jsonString);
```

Object literal = { naam: waarde } → snel object maken

Kan **genest + arrays bevatten**

JSON = tekstformaat voor data (lijkt op object)

Gebruik `JSON.stringify()` en `JSON.parse()`

Opdracht – JSON

Programma 1: object → JSON

```
// programma1.js
const student1 = {
  naam: "Jari Rhellam",
  leeftijd: 19,
  geboortedatum: new Date("2005-03-15"),
  scores: [85, 92, 78],
  adres: {
    straat: "Kerkstraat 12",
    stad: "Kortrijk"
  }
};

const jsonString = JSON.stringify(student1);
console.log(jsonString);
```

Output op console

```
{"naam":"Jari Rhellam","leeftijd":19,"geboortedatum":"2005-03-15T00:00:00.000Z","scores":[85,92,78],"adres":{"straat":"Kerkstraat 12","stad":"Kortrijk"}}
```

Programma 2: JSON → object

```
// programma2.js
const jsonString = '{"naam":"Jari Rhellam","leeftijd":19,"geboortedatum":"2005-03-15T00:00:00.000Z","scores":[85,92,78],"adres":{"straat":"Kerkstraat 12","stad":"Kortrijk"}}';

const student2 = JSON.parse(jsonString);
console.log(student2.naam); // "Jari Rhellam"
console.log(student2.adres.stad); // "Kortrijk"
console.log(student2.scores); // [85, 92, 78]
```

Antwoorden op de vragen

Opdracht contactmangr.

html

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <title>Contactmanager</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<h2>Contactmanager</h2>

<div class="container">

  <!-- Lijst -->
  <label for="personenLijst"></label><select id="personenLijst" size="10"></select>

  <button onclick="nieuwPersoon()">Nieuw</button>

  <!-- Formulier -->
  <div class="form">
    <label>Voornaam</label>
    <label for="voornaam"></label><input type="text" id="voornaam">

    <label>Familiennaam</label>
    <label for="familiennaam"></label><input type="text" id="familiennaam">

    <label>Geboortedatum</label>
    <label for="geboortedatum"></label><input type="date" id="geboortedatum">

    <label>Email</label>
    <label for="email"></label><input type="email" id="email">

    <label># kinderen</label>
    <label for="kinderen"></label><input type="number" id="kinderen">

    <br><br>
    <button onclick="bewaarPersoon()">Bewaren</button>
  </div>

</div>

<script src="script.js"></script>
</body>
</html>
```



```
body {
  font-family: Arial, serif;
  padding: 20px;
}

.container {
  display: flex;
  gap: 20px;
}

select {
  width: 200px;
  height: 200px;
}

.form {
  display: grid;
  grid-template-columns: 150px 200px;
  gap: 5px;
}
```

```
let personen = [];
let huidigeIndex = -1;

// NIEUW
function nieuwPersoon() {
  huidigeIndex = -1;

  document.getElementById("voornaam").value = "";
  document.getElementById("familienaam").value = "";
  document.getElementById("geboortedatum").value = "";
  document.getElementById("email").value = "";
  document.getElementById("kinderen").value = "";
}

// BEWAREN
function bewaarPersoon() {
  let persoon = {
    voornaam: document.getElementById("voornaam").value,
    familienaam: document.getElementById("familienaam").value,
    geboorteDatum: document.getElementById("geboortedatum").value,
    email: document.getElementById("email").value,
    kinderen: document.getElementById("kinderen").value
  };

  if (huidigeIndex === -1) {
    personen.push(persoon);
  }
}
```

```

    } else {
        personen[huidigeIndex] = persoon;
    }

    updateLijst();
}

// LIJST UPDATEN
function updateLijst() {
    let lijst = document.getElementById("personenLijst");
    lijst.innerHTML = "";

    for (let i = 0; i < personen.length; i++) {
        let option = document.createElement("option");
        option.text = personen[i].voornaam + " " + personen[i].familienaam;
        option.value = i;

        lijst.appendChild(option);
    }
}

// SELECT
document.getElementById("personenLijst").addEventListener("change", function () {
    huidigeIndex = this.value;

    let p = personen[huidigeIndex];

    document.getElementById("voornaam").value = p.voornaam;
    document.getElementById("familienaam").value = p.familienaam;
    document.getElementById("geboortedatum").value = p.geboorteDatum;
    document.getElementById("email").value = p.email;
    document.getElementById("kinderen").value = p.kinderen;
});

```